

OTDE Gestão

Estado do sistema, decisões travadas e planejamento

9.753

LINHAS DE
CÓDIGO

172

TESTES
PASSANDO

18

TESTES DE
REGRAS

0–5

FASES
ENTREGUES

Para quê serve este documento: continuar o projeto numa conversa nova sem perder contexto.

Estado do repositório: limpo, nada pendente para enviar. Último commit `66ae1d6`.

Data: 9 de setembro de 2026 · **Projeto:** otdegestao · **Responsável:** Willian

1. O que é o sistema

A OTDE é uma agência que administra em torno de **41 lojas** em marketplaces para cerca de **48 proprietários**. O sistema faz três coisas que antes eram planilha e conversa solta no grupo:

1. **Vigia as lojas todo dia.** Puxa pedidos da Shopee pela API oficial, calcula faturamento, e dispara alerta quando alguma coisa saiu do trilho — prazo de envio, cupom de Prêmio de Seguidor desligado, loja sem sincronizar.
2. **Fecha o mês com o cliente.** O `relatorio-cliente.html` monta a imagem de fechamento que vai para o proprietário, com faturamento, variação e base de cobrança.
3. **Dá tela própria ao cliente e ao especialista.** Cada proprietário vê os produtos dele; cada especialista vê só o marketplace que opera. Essa parte é a mais nova — foi construída nas últimas semanas, nas fases 0 a 5.

A frase que orienta as decisões de segurança do projeto: **o erro caro aqui não é uma tela feia, é um cliente enxergando o custo do outro**. Toda vez que houver dúvida entre conveniência e isolamento, o isolamento ganha.

2. Como o código está organizado

Não tem framework nem build. É JavaScript puro servido pelo Firebase Hosting. O `app.html` carrega os scripts numerados na ordem, e três módulos ESM que se expõem em `window`.

Arquivo	Linhas	O que faz
<code>functions/index.js</code>	1.698	Todo o backend. Cloud Functions v2, região <code>us-central1</code> . Sincronização com a Shopee, criação e remoção de acessos, claims.
<code>relatorio-cliente.html</code>	1.663	Fechamento mensal. Apesar do nome, é tela de funcionário , não do cliente.
<code>js/04-telas.js</code>	1.376	Todas as telas do app: dashboard, lojas, clientes, produtos.
<code>cliente.html</code>	842	Uma página servindo três modos : cliente, especialista e equipe vendo como cliente.
<code>app.html</code>	634	Casca do app, CSS e carregamento dos scripts.
<code>js/08-produtos.js</code>	630	Telas de admin: importação da planilha, SKU, criação de acessos de cliente e de especialista.
<code>js/06-formularios.js</code>	578	Formulários e modais.
<code>js/custos.js</code>	430	Sem consumidor hoje . Ver a nota abaixo.
<code>js/05-clientes-e-acesso.js</code>	409	Cadastro de proprietários, marketplaces, credenciais de loja.
<code>js/planilha-produtos.js</code>	305	Leitor da planilha do cliente, aba por aba. Detecta coluna por conteúdo, não só por cabeçalho.
<code>js/prazos.js</code>	256	Regras de alerta, inclusive o Prêmio de Seguidor.
<code>js/07-interacoes-e-boot.js</code>	255	Boot, roteamento, delegação de eventos.

<code>js/01-estado-e-dados.js</code>	227	Estado global e listeners do Firestore.
<code>js/02-utilitarios.js</code>	171	Formatação, <code>mktsDoCliente()</code> , crachá clicável do proprietário.
<code>firestore.rules</code>	140	Quem lê o quê. É aqui que mora o isolamento entre clientes.
<code>js/03-acesso.js</code>	139	Login e checagem de funcionário.

ATENÇÃO

`js/custos.js` (430 linhas, com testes) **não tem nenhum consumidor**. O `app.html` importa e expõe em `window.custos` , mas nenhuma linha lê isso — é peso morto baixado em todo boot. Eu já afirmei antes que a conferência diária usava esse arquivo; **não usa**. O leitor de planilha que está em produção é o `js/planilha-produtos.js` , escrito depois e desenhado contra uma planilha real de cliente. Antes de mexer em importação, é o `planilha-produtos.js` que importa.

3. Modelo de dados

Coleções do Firestore que interessam para a área do cliente:

```
accounts/{uid}
  papel      "cliente" | "especialista" | "equipe"
  custId     id do proprietário          (só cliente)
  mkt        "Mercado Livre" | "TikTok"   (só especialista)

products/{cust__chave}
  custId · sku · nome · custo · preco · margem · lucro
  custNome      denormalizado, para a regra e a tela
  mkts[]        denormalizado, é o que o especialista filtra
  peso · medidas {c,l,a} · fotos[] · video   (links do Drive)
  criadoEm · criadoPor {uid, emNomeDe}

listings/{loja__anuncio}
  custId · sku · mkt · storeId · itemId · preco · status
  custNome · storeNome · storeMkt · mkts[]   denormalizados
```

Por que tanto campo repetido

Os campos `custNome`, `storeNome` e `mkts[]` estão duplicados de propósito. Regra do Firestore não faz *join*: para o especialista ver "os produtos dos marketplaces que ele opera", o marketplace precisa estar **dentro do documento do produto**. Sem isso, cada leitura vira um `get()` dentro da regra — que é leitura cobrada e latência em cima de cada item da lista.

Ids determinísticos

Nada usa id aleatório: produto é `cust__chave`, anúncio é `loja__anuncio`, especialista é `esp_<mkt>__<produtoId>`. Consequência prática: **reimportar a mesma planilha atualiza, não duplica**. É o que permite o cliente mandar a planilha corrigida sem virar bagunça.

4. Quem enxerga o quê

O papel vive em `accounts/{uid}` e é espelhado nas **custom claims** do Firebase Auth. As regras leem a claim:

```
request.auth.token.get("papel", "") // não erra quando a claim não existe
request.auth.token.get("custId", "")
request.auth.token.get("mkt", "")
```

O `.get(campo, "")` não é detalhe de estilo: token sem a claim **nega** em vez de **dar erro**, e erro em regra some com a tela inteira.

Papel	Lê	Não lê
cliente	seus <code>products</code> e <code>listings</code> (<code>custId</code> igual ao da claim)	qualquer coisa de outro proprietário
especialista	<code>products / listings</code> cujo <code>mkts[]</code> contém o <code>mkt</code> dele	marketplace que não opera
equipe / admin	tudo, via <code>employees/{uid}</code>	—

ATENÇÃO

Nenhum papel externo lê `clients` ou `customers`. Não é esquecimento e não deve ser "relaxado" para facilitar uma tela. `clients.access` guarda a **senha do marketplace da loja**, e regra do Firestore não esconde campo: ou libera o documento inteiro, ou não libera. O nome do proprietário chega às telas por `custNome` denormalizado, justamente para não precisar abrir esse documento. Está escrito em comentário dentro do `firestore.rules` — mantenha o comentário.

5. O que já está entregue

Fase 0 — Contas e papéis pronta

`accounts/{uid}`, funções `criarAcesso` e `removerAcesso`, claims aplicadas. 18 testes de isolamento no emulador provam que cliente lendo outro cliente e especialista lendo marketplace alheio **falham**. `2fcd4c4`

As claims são aplicadas por uma **função comum** chamada por quem grava (`aplicarClaims()`), **não** por gatilho do Firestore. O gatilho existia e foi removido: dependia do Eventarc, quebrava o deploy com erro de propagação de permissão e insistia em nascer em `southamerica-east1`. Não recoloque o gatilho achando que é mais elegante. `4420460`

Fase 1 — Marketplaces de cada cliente pronta

Caixinhas no cadastro do proprietário. `mktsDoCliente()` trata lista vazia como "ninguém preencheu ainda" e deduz pelas lojas, em vez de tratar como "não opera em nenhum".

Fase 2 — Importação da planilha pronta

Tela de admin: escolhe o cliente, cola a aba, vê quatro contadores (entrou / conflito / vazia / sem identidade) e confirma. Conflito de custo aparece com "deixar de fora" pré-selecionado. `769afa3` · `efffda5`

- Coluna é achada pelo cabeçalho **e pelo conteúdo**: se 60% dos valores forem URL, a coluna é de link; entre link de marketplace e link de mídia, decide pela contagem.
- Marketplace é reconhecido pelo **host** da URL, não pela URL inteira.
- **Lucro e margem vêm da planilha, nunca são recalculados**. Preço menos custo dá 52% onde o valor real é 7,5% — porque a planilha do cliente já desconta comissão, frete e imposto. Quando a planilha não traz, a tela mostra "ainda não informada".
- Campo vazio na reimportação **não apaga** o que já estava gravado.

Fase 3 — SKU do marketplace pronta

O SKU **vem do marketplace**, não da aba de SKU da planilha. Ele já estava guardado em `sales.items` (`s / m / i`) — não precisou de integração nova, só de costura. Uma tela de casamento por dicionário chegou a ser construída e foi **descartada**: só 2 de 19 nomes casavam entre as abas e a nota de semelhança não previa acerto. `cc5f3fd`

Fase 4 — Área do cliente pronta

`cliente.html`, página própria e não aba do app — o app abre listeners em tempo real de 41 lojas no boot, e o cliente não precisa carregar isso. `2852f32` · `a5bc42a`

Fase 5 — Área do especialista pronta

Mesma página, filtrada pelo `mkt` da conta. `02173ac`

Entrar como cliente, com autoria dupla pronta

Não usa token de personificação: **muda a tela, não a sessão**. O funcionário abre `cliente.html?cliente=...` e só passa depois de checar `employees/{uid}`. Toda gravação carimba `criadoPor` (quem digitou) e `emNomeDe` (por quem). O admin entra pela barra lateral, em  **Produtos**, com o cliente embutido num iframe — vê exatamente o que o cliente vê. 9fea7e7

6. Decisões travadas

Foram fechadas com você e o sistema inteiro está construído em cima delas. Reverter qualquer uma custa retrabalho grande — não mude sem intenção.

Decisão	Consequência
Foto e vídeo no Google Drive	Não existe Firebase Storage nem custo de armazenamento. O sistema guarda link , não arquivo. Pendência conhecida: o arquivo precisa estar como "qualquer pessoa com o link", senão a imagem quebra para quem não é o dono.
Mercado Livre e TikTok pela API de cada marketplace	Vira fase própria e grande (fase 7). Não bloqueia nada antes dela — o ML já foi destravado na fase 5 sem depender da API.
Especialista vê só o que opera	Regra mais estreita e o custo não circula além de quem precisa dele para precificar.
Produtos importados das planilhas	A integração de catálogo da Shopee sai do caminho crítico . Vira melhoria futura, para pegar o produto que nunca vendeu — porque a planilha é registro de vendas, e o que nunca vendeu pode não estar nela.
Claim em vez de <code>get()</code> na regra	Cada <code>get()</code> numa regra é leitura cobrada e latência em cima de cada item da lista.
Ids determinísticos	Reimportar atualiza em vez de duplicar.
Margem e lucro vêm da planilha	Recalcular mente por um fator de sete.
Autoria dupla em toda gravação	Dá para saber depois se foi o cliente ou a equipe que preencheu.

7. O que ficou pendente

Para fazer assim que sentar **pendente**

- Reimportar as duas abas da Shopee.** Os anúncios importados antes da última versão não têm `margin`, `lucro`, `storeNome`, `mkts` nem `custNome`. **Sem esses campos a tela do especialista aparece vazia** — e vazia parece bug, não parece dado faltando.
- Rodar `TESTAR-REGRAS.command`** (18 casos) antes de publicar a liberação da leitura de `listings`. Depois `PUBLICAR.command`.
- Criar a conta do especialista de Mercado Livre** em  **Produtos** →  **Especialistas**.
- Redigitar e salvar o 7,21** em "Variação mês passado" da EVA HOME. Aquele valor foi gravado *antes* da bandeira `varAntManual` existir, então o registro antigo ainda é tratado como automático.
- Confirmar qual loja recebeu a aba "Laços de Lã"**. No sistema, "Laços de Lã" está cadastrada como **Shein**, mas a loja da Shopee chama "LAÇOS DE LÃ MODA BEBÊ". Pode ser que a aba tenha entrado na loja errada.

Perguntas em aberto

Cada especialista tem uma conta única no marketplace, com todos os clientes dentro? Essa pergunta ainda não foi respondida e ela decide o desenho da fase 7: se for uma conta só, **o dono do anúncio não sai da conta — sai do SKU**, e a integração precisa ser construída em cima do SKU desde o primeiro dia.

Buraco conhecido

O pop-up "**Quero anunciar**" na área do cliente mostra a mensagem e **não registra nada**. O cliente acha que pediu; ninguém recebe o pedido. Ou vira registro em coleção própria, ou o texto tem que deixar claro que o pedido é pelo grupo.

8. Armadilhas — coisas que já custaram tempo

O padrão "o automático passa por cima de quem digitou"

No `relatorio-cliente.html` apareceram **três** campos com exatamente o mesmo defeito: você digitava, salvava, e o valor calculado voltava por cima. Variação do mês atual, variação do mês passado e faturamento dos últimos 30 dias. `9b6555e` · `3d56b16` · `66ae1d6`

Os três foram corrigidos com a mesma forma, e é essa forma que deve ser usada se aparecer um quarto caso:

- **Campo vazio não é zero.** String vazia vira `null`, não `0`.
- Uma bandeira explícita por campo (`varAtuManual`, `varAntManual`) gravada junto.
- No desenho, **o manual vence**; o automático só entra quando a bandeira é falsa.
- Registro antigo, sem a bandeira, continua automático — não quebra o histórico.
- O `fillForm` devolve ao campo **só o que foi digitado**, nunca o calculado.

ATENÇÃO

Se aparecer um quarto campo com esse comportamento, **não conserte um só** — vale varrer a tela inteira de uma vez procurando o mesmo padrão.

Botão em modal não clica

O app usa **delegação de eventos** num listener preso ao `#content`. Modal vive **fora** do `#content`. Botão novo dentro de modal precisa ser registrado no `rebind()` daquele modal, senão o clique simplesmente não acontece — sem erro no console, sem pista.

A classe de modal do projeto é `.form-panel`

Não existe `form-modal-header`, `form-modal-body` nem `modal-close` no `app.html`. Eu inventei essas classes uma vez e as telas saíram sem estilo nenhum. Antes de escrever CSS novo, procure a classe que já existe. `e921e05`

O macOS tem um Java falso

`command -v java` retorna verdadeiro no macOS mesmo sem Java instalado — existe um **stub** em `/usr/bin/java` que só serve para sugerir a instalação. A checagem certa é `java -version >/dev/null 2>&1`, mais `/usr/libexec/java_home` e os caminhos *keg-only* do Homebrew. Sem Java, o emulador do Firestore não sobe e os 18 testes de regras não rodam. `5c84eaf` · `e01b960`

Mensagem de commit quebra o shell

Aspas e crases dentro de `git commit -m` viram execução de comando — uma vez uma crase em torno de `java` executou o programa e injetou o texto de ajuda dentro da mensagem. Use sempre `git commit -F arquivo`.

Arquivos de trava do git

O acesso aos arquivos do Mac por esta conversa **não apaga arquivo**. Quando sobra um `index.lock` ou `HEAD.lock`, o jeito é mover para `.git/_to_delete/` — e mover **imediatamente antes** de cada comando git, não no começo da sessão.

O token de sincronização não entra em URL de navegador

O `SYNC_TOKEN` nunca deve aparecer em endereço aberto no navegador. Os diagnósticos rodam pelos `.command`, que leem o token do ambiente.

Diagnóstico de cupom: o id da loja não é o número da URL

O `amostraCupons` devolveu `porLoja: []` porque recebeu o número que aparece em `shopee.com.br/shop/1386095840/`, e não o id do documento no Firestore. O `CUPONS-DIAGNOSTICO.command` hoje lista as lojas e você escolhe pelo número da lista.

Prêmio de Seguidor

O campo foi descoberto em dados reais da API, não chutado:

```
export const PREMIO_SEGUIDOR = { campo: "voucher_purpose", valor: 3 };
```

E tem guarda para não acusar loja que ainda não sincronizou: se nenhum cupom trouxe dados brutos, o sistema **não afirma** que o prêmio está desligado. `c22b134`

9. Como testar e publicar

Comando	O que faz
<code>node --test testes/*.test.js</code>	172 testes em 7 arquivos. Roda rápido, sem emulador.
<code>TESTAR-REGRAS.command</code>	18 casos de isolamento no emulador do Firestore. Precisa de Java e só roda no Mac — o container da conversa não consegue baixar o emulador.
<code>PUBLICAR.command</code>	Testes → sintaxe → carimbo de versão (quebra de cache) → homolog → confirmação → produção. Publica <code>functions</code> , <code>hosting</code> e <code>firestore:rules</code> .
<code>CUPONS-DIAGNOSTICO.command</code>	Amostra de cupons de uma loja escolhida em lista.
<code>REVERTER-se-quebrar.command</code>	Volta a publicação anterior.

ATENÇÃO

Regra que já se provou cara: **mexeu em `firestore.rules`, roda `TESTAR-REGRAS.command` antes de `PUBLICAR`**. Regra errada em produção ou derruba a tela de todo mundo, ou abre o custo de um cliente para outro — e a segunda não dá aviso nenhum.

10. O que vem depois

Fase 6 — Fechar os buracos das fases 0 a 5

- Registrar o pedido do "Quero anunciar" em algum lugar.
- Avisar quando o link do Drive não está público, em vez de mostrar imagem quebrada.
- Reimportação em lote das abas já enviadas, para preencher os campos denormalizados.

Fase 7 — APIs do Mercado Livre e do TikTok a maior

Duas integrações independentes, cada uma com cadastro de aplicativo, autorização e renovação de token. Ficou por último de propósito: é a única parte do projeto que **depende de aprovação de terceiro**, e o prazo não está na sua mão. Antes de começar, responda a pergunta da seção 7 sobre conta única por especialista — ela muda o desenho inteiro.

Melhoria adiada — catálogo da Shopee pela API

Hoje o backend lê **pedidos**, não produtos. Puxar o catálogo passaria a incluir o produto que nunca vendeu, que a planilha não traz. Deixou de ser urgente quando os produtos passaram a vir da planilha.

11. Como retomar numa conversa nova

Comece dizendo o que quer fazer e anexe este PDF. As três coisas que a conversa nova precisa saber antes de escrever qualquer linha:

1. O projeto está em `otdegestao`, no Mac, e o repositório está limpo em `66ae1d6`.
2. As decisões da seção 6 estão travadas — não são sugestões a reavaliar.
3. As armadilhas da seção 8 já custaram tempo uma vez cada. Não vale a pena descobrir de novo.

OTDE Gestão · documento de continuidade gerado em 9 de setembro de 2026 · base: commit 66ae1d6 · 9.753 linhas · 172 testes + 18 testes de regras.